

填空题

1. CPU调度牵涉到的进程状态转移 中的 从Running到Terminated、从Running到Waiting、从Running到Ready、从Waiting到Ready
 - 填空题不一定是短空
 - 抢占性牵涉到 从Running到Ready、从Waiting到Ready
2. 计算机系统保护包括哪四个部分：
 - 二态模式、特权指令、CPU保护、内存保护
3. 用户线程与内核线程的三种关系 --- 一对一、多对一、多对多
4. 银行家算法 Request 时一定注意是哪个进程，特别是 P0 和 P1
5. 静态优先级调度的问题 --- 饥饿 和 优先级反转。 解决方法 -- 老化
6. **产生优先级反转： SJF、多级队列**
7. 页表结构
 - 层次页表，哈希页表，反向页表
8. 虚拟内存实现：
 - 需求分页
 - 需求细分
9. 内存地址描述 --- 合法， 非法
10. 逻辑地址，物理地址转换 --- 无效转换
11. ACL --- access control list
12. 上下文切换： 操作系统从一个进程切换到另一个进程执行
13. PTE是invalid时，表示逻辑地址非法 或者 合法但是未加载到内存中
14. FAT文件系统采用 **链接分配**
15. 文件系统空闲空间管理 --- bit向量、链表
16. **资源分配图的请求箭头指向方框；分配箭头从方框内资源指出**
17. 解决竞争条件必须满足
 - 互斥
 - progress(前进性)
 - 有限等待
 - 速度
18. 死锁的四个必要条件
 - 互斥
 - 保留并等待
 - 无抢占
 - 循环等待
19. 三种常见内存分配方法：
 - 首次适应
 - 最佳适应
 - 最坏适应
20. CPU调度算法：
 - FCFS
 - SJF

- SRTF
- 优先级
- RR
- 多级队列
- 多级反馈队列

21. 二元信号量到计数信号量 -- 16-17卷子考了

22. FAT是典型的链接分配文件系统

选择题

10、关于虚拟内存，哪一个是正确的？（ ）

- (A) 虚拟内存只能通过请求式调页（demand paging）实现。
- (B) 虚拟内存是一个纯软件的系统，不需要硬件的支持也能实现。
- (C) 如果一台 32 位的计算机拥有 4GB 的物理内存，那么它永远都不会发生“颠簸（thrashing）”。
- (D) 在虚拟内存中，有效内存访问时间近似正比于缺页异常发生的概率。

D

1、进程同步的经典问题包括（ ）

- (A) 读者写者问题
- (B) 拜占庭问题
- (C) 哲学家进餐问题
- (D) 皮特森问题
- (E) 有限缓冲区问题

ACE

3、进程调度可分为（ ）

- (A) I/O 调度 (B) 资源调度 (C) 长期调度 (D) 短期调度 (E) 中期调度

CDE

10、下列属于文件基本操作的是（ ）

- (A) 截断 truncate
- (B) 读取 read
- (C) 写入 write
- (D) 创建 create
- (E) 删除 delete

ABCDE

最后一道大题

- 第一步： 计算一个页面放多少个向量
 - 一个页面可能存放多个向量，也可能多个页面存放一个向量
- 物理地址不要抄错了， frame1 抄成 frame0 -- frame0存的代码
- 逻辑地址时 注意code对应0000, a[0]对应的0001
 - 两道大题.pdf -- 第二题
 -
- 内存访问次数
 - 无 TLB --- 有无page fault 都是 2次 内存访问
 - 有 TLB
 - hit → 1 次内存访问, 0 page fault
 - miss → 2 次内存访问, 1 page fault
- 时刻注意 i 、 j 从 0 还是 1 开始
- 逻辑地址 = page number + page offset

第一章 Intro

重点：CS结构、OS组成、OS历史、UNIX例子、实时类别

- **计算机系统的结构**：硬件→操作系统→系统和应用程序（编译器，汇编）→用户
- 操作系统是什么？
 - 控制程序，资源管理器，扩展机器
- **操作系统组成(component)**
 - CPU管理器、内存管理器、文件系统、设备管理器
- 操作系统历史
 - 批处理系统、多道程序系统、分时操作系统、OS for 桌面机 和 嵌入式系统
 - 注意区别前3个
 - 批处理：一个时间点只能运行一个
 - 多道程序：多个job在 内存 中，CPU复用
 - 分时：job存储在 内存和磁盘，只有内存中的job才能上CPU
- 桌面操作系统
 - MS-DOS, Windows NT/XP
 - **UNIX** --- BSD UNIX、Solaris、Mac OS
 - Linux
- 嵌入式操作系统
 - 关键词： 控制设备(control device) 有限资源
 - 可能有 **时间要求**，即**实时性**(Real time)
 - 硬实时：在特定的时间点动作必须出现

- 软实时： 偶尔错过时限要求可以接受

第二章 计算机系统结构

- 整体结构
 - CPU和总线相连，其他如存储器、设备等 通过各自的控制器与总线相连
 - CPU和设备控制器 并发执行，竞争cycles。
 - 通过内存控制器实现 并发内存访问
- Bootstrap(引导)
 - 将Kernel代码从 持久存储 加载到 内存中，并将控制转到OS入口。---- Bootstrap
 - 执行Bootstrap的程序叫做 **boot-loader(引导器)**
- 中断(Interrupt)
 - 关键词： **中断驱动**、IRQ(Interrupt Request)、IVT、IV(ISR地址)、ISR
 - 中断响应牵涉 --- 硬件执行 (IP, SP, FLAGS) 和 ISR(汇编和C)
 - 中断出现 → 硬件执行 → 通过IRQ查询IVT找到对应的IV，跳转到ISR，执行ISR
- 异常(Exception)
 - **异常--同步；中断--异步，外围设备触发**
 - 异常：指令执行过程中检测到错误。 **指令错误** 除零错误，内存访问地址非法
 - 别称： **SGI (Software generate Interrupt)** 和 **同步中断**
- 如何访问外围设备
 - CPU通过 **设备控制器** 访问设备
 - 设备控制器 有两种寄存器： 命令寄存器 和 数据寄存器
 - 访问方式
 - **I/O 端口 IN/OUT** --- I/O指令，都是**特权指令**
 - **内存映射 I/O** --- 用访问内存的方式（即内存地址）访问设备寄存器。
 - 可寻址地址空间必须预留一部分作为 Memory map
 - 使用**相同的总线**来寻址内存和 I/O 设备
 - 同时使用： **命令传输用I/O port；数据传输用 memory mapped I/O**
- 硬件保护
 - 方式： **二态模式 特权指令 CPU保护 存储器保护**
 - 二态模式： **用户态 和 内核态 模式位(mode bit) 1=user**
 - 特权指令
 - 所有的I/O指令都是特权指令
 - 特权指令只能在 内核态 使用
 - 特权指令在用户态使用 触发 异常
 - CPU保护
 - Timer --- 定时器中断 --- 计数寄存器
 - 存储器保护 --- 确保内存访问地址合法，确定**合法物理内存地址范围**
 - base, limit register
 - $\text{logical address} \geq \text{base} \rightarrow \text{logical address} < \text{base} + \text{limit}$
- 系统调用(system call) --- OS与用户程序之间的**接口**
 - 系统调用是在 **内核态**

- 用户程序**只能**通过系统调用使用OS提供的服务
- 系统调用过程
 - 参数准备 -- Wrapper -- system call number in register -- trap into OS -- index system call SR in SC table -- 执行服务例程 -- 返回用户程序
- 陷入OS
 - **异常(int 21h) 或者 特殊指令(SYSENTER/SYSEXIT)**
- 系统调用与库函数
 - 系统调用陷入OS, 运行更慢; 库函数不陷入OS
 - 用户程序可以直接系统调用, 也可以通过库函数进行系统调用

第三章 操作系统结构

- OS结构
 - 简单结构 -- MS-DOS, UNIX
 - 分层结构 -- THE, IBM OS/2
 - 难以划分每层
 - 低效
 - 微内核 -- Mach, Windows XP/NT
 - 微内核: **CPU管理器, 内存管理器, 通信设备**
 - 虚拟机 -- 硬件虚拟机: VM370, XEN, KVM, VMware --- 软件虚拟机: JVM
 - 虚拟机实现: 仿真--VMware 半虚拟化--Xen 本机虚拟化--KVM, VM370
 - 虚拟机对系统资源提供了全面保护; 虚拟机之间不允许直接共享资源
- OS设计
 - 设计目标: 用户目标和系统目标
 - **策略(what) 与 机制(how) 分离**

第四章 进程管理

- 进程: 就是运行中的程序。对比程序, 含有资源
 - **组成:** 文本段, 数据段--全局变量, 寄存器, 堆--用于动态分配的内存, 栈--局部变量, 函数, 参数
- **进程状态:** new, ready, running, waiting, terminated
- **CPU调度 与 抢占式 对应的状态变迁** --- 最上面第1点
- **PCB(Process Control Block)**
 - 进程号 PID
 - 进程状态
 - 寄存器保存区 PC
 - 内存管理信息
 - 记账信息
 - IO状态信息
 - 调度信息
- 进程调度三个关心
 - what happened now:

- **上下文切换：从一个进程切换到另一个进程执行**
- 调度前保存上下文，运行前恢复上下文
- 瓶颈
- how trace
 - **作业队列，就绪队列，IO设备[等待]队列**
- who next:
 - **CPU调度器 调度算法**
- 进程的操作：
 - 创建 -- **fork in UNIX, CreateProcess in Windows,**
 - 终止 -- 正常 **exit in UNIX;**异常 **kill in Unix**
 - 通信
 - 协作 -- 需要同步动作
- 进程协作优点： **信息共享、计算加速、模块化**
 - 协作要求： **通信和同步** --- IPC实现
 - **缓冲区**
- **IPC (Interprocess communication)**
 - 允许进程通信和同步，不需要共享相同的地址空间
 - 消息传递，管道，套接字
- 管道
 - 匿名管道 -- **单工，父子进程间，进程结束管道消失**
 - **fd[0]**是读口

第五章 线程管理

- **进程是资源分配的单元；线程(thread、LWP-lightweight process)是CPU调度分发的实体**
- 多线程
 - **共享文本段和数据段**
 - 每个线程有私有的 **线程上下文(寄存器集和状态信息)** 和 私有的**栈段**
- 线程优点：
 - **可响应性、资源共享、经济、多处理器架构利用**
- 用户线程与内核线程 --- **一对一，多对一，多对多**
- 多线程编程接口 --- **Pthreads -- POSIX 标准 -- API of 线程创建和同步**

第六章 CPU调度

- **调度是操作系统的基本功能**
- CPU调度的成功取决于观察到进程有如下属性：
 - 进程执行由 **CPU执行-IO等待** 周期组成
 - 进程在这两种状态间转换
- **CPU调度 与 抢占式调度 对应的状态变迁** --- 最上面第1点

- **调度标准** -- 5个
 - CPU利用率
 - 吞吐率
 - 轮转时间
 - 等待时间 = 结束 - 到达 - 执行
 - 响应时间
- 调度算法
 - FCFS
 - **SJF(最少平均等待时间)** -- SRTF
 - 优先级调度 -- 静态和动态优先级
 - 静态优先级调度问题: **饥饿、优先级反转** --- 解决: **老化**
 - **SJF和多级队列都会引起优先级反转**
 - 轮转调度(Round-Robin RR) -- 时间片q 很大→ FCFS
 - 多级队列 -- 按优先级分队列, 每个队列调度算法可以不同
 - 多级反馈队列 -- 优先级越高, 时间片越小; 时间片用完, 优先级下降 -- **下沉**

第七章 进程同步

- 竞争条件是多个进程**同时访问和操作相同数据**的情况, 执行结果取决于**访问发生的特定顺序**
- 竞争条件避免
 - 禁止同时 访问和操作 共享数据
 - 访问必须 **序列化**
- **临界区: 访问共享资源的代码**
 - 进入区, 临界区, 退出区, 剩余区
- **解决竞争条件必须满足**
 - 互斥
 - 前进性(progress) -- 空闲让进
 - 有限等待
 - 速度 -- 对 CPU速度和数量无要求
- 解决竞争条件:
 - 软件实现 -- (turn; flag) --不满足前进性; flag+turn-Peterson算法
 - turn -- do {while(turn ≠ i); Enter; turn=j;} -- 别人来给自己
 - flag 和 peterson -- 自己给自己权限 -- 在 do while 内修改
 - 硬件实现 -- 关中断(CLI STI); 特殊指令(TSL 或者 SWAP)
 - 关中断 -- (CLI STI)特权指令 -- **用户进程不允许关中断**
 - `bool TSL(bool &target){bool rv=target; target=true; return rv;}`
 - `void SWAP(bool &a, bool &b){bool temp=a;a=b;b=temp;}`
 - **TSL 和 SWAP 都是 原子操作 都不满足有限等待**
 - 这些解决都有一个问题: **忙等** -- 这种类型的解决叫 **自旋锁**
 - 一个进程在临界区中, 其他想进入临界区的进程在进入区陷入持续循环
 - 自旋锁 只对 **多处理器系统有用**; 没有上下文切换--没下CPU
- **信号量(semaphore)** -- 整型变量, 代表资源数量

- 信号量除了 `init`; 只被两个 **原子操作 P 和 V** 访问

- **原子操作实现** --- 关中断 in 单处理系统; 自旋锁 in 多处理器系统

- ```
1 struct{
2 int value;
3 struct PCB* L;
4 }semaphore;
5
6 void P(semaphore* S){
7 S → value --;
8 if(S→value < 0){
9 add (currProcess) to S→L;
10 currProcess→state = WAITING;
11 scheduler();
12 }
13 }
14
15 void V(semaphore* S){
16 S→value ++;
17 if(S→value ≤ 0){
18 remove a process (A) from S→L;
19 A→state = READY;
20 }
21 }
22
```

- **生产者 消费者** --- 先同步, 再互斥

```
1 semaphore mutex, empty, full;
2 mutex.value = 1; // 保护临界区
3
4 // 同步
5 empty.value = BSIZE;
6 full.value = 0;
7
8 void Producer(){
9 do{
10 Produce an item;
11
12 P(&empty);
13 P(&mutex);
14
15 Critical Section;
16
17 V(&mutex);
18 V(&full);
19 }while(1);
20 }
21
22 void Consumer(){
23 do{
```

```

24 P(&full);
25 P(&mutex);
26
27 take an item from Critical Section;
28
29 V(&mutex);
30 V(&empty);
31
32 consume;
33 }while(1);
34 }

```

## • 读者-写者

```

1 semaphore mutex, wrt;
2 mutex.value = 1; // 保护 readcount
3 wrt.value = 1; // 保护共享数据
4
5 int readcount = 0;
6
7 void Writer(){
8 do{
9 P(&wrt);
10
11 write to buffer;
12
13 V(&wrt);
14 }while(1);
15 }
16
17 void Reader(){
18 do{
19 P(&mutex);
20 readcount ++;
21 if (readcount == 1)
22 P(&wrt);
23 V(&mutex);
24
25 read from buffer;
26
27 P(&mutex);
28 readcount --;
29 if (readcount == 0)
30 V(&wrt);
31 V(&mutex);
32 }while(1);
33 }

```

## • 二值信号量

```

1 struct{

```

```

2 bool vaule;
3 struct PCB* L;
4 }bisemaphore;
5
6 void P(bisemaphore* S){
7 if(S→value == true)
8 S→value = false;
9 else{
10 add (currP) to S→L;
11 currP→state = WAITING;
12 scheduler();
13 }
14 }
15
16 void V(bisemaphore* S){
17 if (S→L == empty)
18 S→value = true;
19 else{
20 remove a process (A) from S→L;
21 A→state = READY;
22 }
23 }

```

- **高级语言同步构造 -- 管程** -- 过程、变量、数据结构的集合
  - 管程实现互斥的性质: **任何时刻管程中只能有一个进程处于活跃状态**
  - **条件变量** --- 仅有 wait()-P 挂起进程 和 signal()-V 唤醒进程
    - signal()不像V操作, **当没有挂起进程时, signal()不会改变什么**--无影响;而V会使得 value++;
  - P 唤醒, Q处于挂起 -- 两个进程同时在管程
    - Hoare--唤醒并等待--P挂起, Q运行
    - Mesa--唤醒并继续--P运行, Q挂起 -- Java
    - Brinch-Hansen--P立即离开管程、
  - **信号量和管程 功能等价, 但是使用和实现不同。两者可以相互实现。**

## 第八章 死锁

- 死锁: 集合中的每个进程都在等待只有集合内其他进程导致的事件
- **使用资源的动作: 请求、使用、释放**
- 死锁4个**必要条件**:
  - 互斥
  - hold and wait(保留并等待)
  - 无抢占
  - 循环等待
- 死锁描述 --- **资源分配图**

- 无环路, then 无死锁
- 有环路, 可能死锁
- 解决死锁: **死锁预防、避免、检测与恢复、鸵鸟算法**
  - 死锁预防 -- 打破必要条件
    - 互斥 -- 将非共享资源转换为共享资源--对设备进行**假脱机**
    - 保留并等待 -- 请求时不拥有资源; 执行前拥有所有资源
      - 结果: **资源利用率低 或者 饥饿**
    - 无抢占 -- 如果进程请求无法获得的资源, 那么自己拥有的所有资源也要释放。
    - 循环等待 -- **对资源类型进行总排序**, 进程按照**枚举的递增顺序**进行资源请求
  - 死锁避免 -- **银行家算法(精确记忆)** -- 分配资源前进行, 避免进入不安全状态
    - Max, Allocation, Available, Need
    - 安全状态 -- 没有死锁; 不安全状态 -- 可能死锁
    - used in THE ---  $m \times n^2$  动作
  - 死锁检测与恢复--精确记忆
    - Allocation、Request、Available
    - 死锁检测 -- 类似银行家算法
    - 恢复 -- 进程终止(所有死锁进程; 一次终止一个进程直到没有死锁); 资源抢占
  - 鸵鸟算法 -- UNIX/Linux; Windows

## 第九章 内存管理

- 内存管理方法: **覆盖和交换**
  - 交换--进程换出到后备存储。换进时必须在相同的内存地址处
- 内存管理问题: **重定位、内存管理、内存分配**
- 重定位: **编译时、加载时、执行时 绑定**
- 地址绑定: 把程序中的符号映射为地址
- 几个概念
  - 逻辑地址: CPU产生的地址
  - 物理地址: 系统中内存单元看到的地址
  - MMU(memory management unit) 实现逻辑地址到物理地址转换的硬件
- 内存保护
  - 对于CPU地址(像是物理地址) --- base、limit 寄存器
  - 对于逻辑地址 --- limit、重定位寄存器
- 内存分配
  - 可用内存块 -- hole(孔) -- 分配, 释放并邻近合并
  - **外部碎片**: 留下的hole不足以满足任何进程的需求, 且不能和邻近的hole合并
  - **内部碎片**: 分配给进程的内存比要求的多
  - 分配算法: 连续物理地址空间分配---**首次适应, 最佳适应, 最坏适应**
    - 首次适应、最佳适应---外部碎片 最坏适应--内部碎片
  - 解决外部碎片---紧缩(compaction) 或者 **非连续物理地址空间分配(分页, 分段, 段页式)**
  - 分页优点:
    - 非连续的物理内存分配
    - 没有外部碎片, 有一定的内部碎片

- 对进程的部分内存进行换入换出
- **硬件和操作系统共同实现**
- PTE -- page table entry
- 由于是操作系统在控制地址映射，它必须
  - **记录系统物理内存的使用状况**。一般使用一个称为**frame table**的数据结构来保存系统中每一个frame的状态；
  - **为每一个进程保存一个page table**
- **page table 必须在内存中** -- PTBR、PTLR寄存器 记录页表位置信息
- 无TLBs: **一次内存访问两次内存操作** --- **page table 和 内存**
- 有TLBs(translation look-aside buffers): 内存操作-- 命中-1次; miss-2次
  - **但是不管命中不命中，都有一个访问TLBs的时间**
  - $EAT = (1 + \epsilon) \times \alpha + (2 + \epsilon) \times (1 - \alpha)$  a: 命中率
- 分页内存保护: PTE含有 **保护信息RWX(只读, 读写, 执行)** valid/invalid
- 一个frame/page 装不下 整个pagetable → 页表结构: **层次、哈希、反向 页表**
- 层次型页表--- 外部页表(页表目录directory) PDE
- **帧管理** --- **空闲帧链表**
- 以 **页面为单位** 进行内存共享
- 分段: **不固定大小的段，每个段存储一种类型的数据** 产生外部碎片
  - logical address = <段号, 段偏移>
  - 段表 -- STBR, STLR寄存器保存段表位置信息
  - STE = base, limit, **保护位(RWX)**, 不像PTE是帧号
- **IA-32 段页式存储典范**
  - STE 被称为 **描述符**
  - 先分段的方式获得地址，在对这个地址用分页的方式查找物理地址

## 第十章 虚拟内存

- 虚拟内存提供了 **巨大的，连续的逻辑内存**
- **只将程序执行需要的部分 放入内存中** -- lazy swapper
- 虚拟内存实现通过:
  - **需求分页，需求细分**
- valid -》地址合法 且 相关页在内存中
- 地址转换查看页表发现是invalid; CPU触发 **缺页异常**; (后面的都是 **缺页异常服务例程**) OS检查 内部表 查看是否是合法内存访问; 非法就终止进程; 合法就检查有无空闲帧; 有就直接将页面从 **交换空间(辅助存储器)** 读入 **内存**; 没有就进行页面置换; 返回中断的指令处继续执行
- 多道程序 → **物理内存的过度分配**
- 页面置换: 没有空闲帧时, 选择一个 **受害者帧**, 根据 dirty bit 选择是否将其写回交换空间; **修改页表--变为无效**; 置换; **修改页表**; restart
- **有空闲帧，一次修改页表; 没有空闲帧，两次修改页表**
- 页面置换算法:
  - FIFO-Belady 异常, optimal, LRU, 二次机会 (访问位)
- **颠簸(thrashing)**: 进程换页所用时间多于CPU执行时间。频繁的换页

- 物理帧数少于进程最低要求-》频繁的换页-》低CPU利用率-》os认为CPU空闲，增加多道程序程度-》换页更频繁了
- 缓解颠簸：**降低多道程序程度**
- why thrashing: **所有进程的局部之和 > 总的内存大小**
  - 局部模型：局部是一个进程在一段时间内频繁使用的页面集合
- **工作集模型** -- 是局部模型的近似 **避免了thrashing，同时保持高的多道程序程度**
  - $\triangle$  考虑t时刻以及以前 多少个页面
  - $WSS_i$  进程i的工作集(频繁使用的页面)
  - OS为每个进程模拟工作集，并为每个进程分配 **工作集大小的 帧数**

## 第十一章 文件系统

- 文件--逻辑连续的辅助存储
- 文件操作：创建，打开，关闭，读，写，搜索，删除
- **决定文件类型的是文件内容**
- 文件访问 -- 顺序访问，随机访问
- **文件系统驻留在设备上**，设备被拆分为文件系统所在的一个或多个分区
- 目录结构：单级目录，二级目录(用户)，树结构，无环图(相同文件不同名字，eg桌面图标)
- 文件系统挂载 (mounted)
  - **驻留在设备上的文件系统必须挂载后才能访问。**
  - 挂载文件系统的位置称为**挂载点**。
  - 通常，挂载点是一个空目录。
- UID, GID 三类用户 owner, group, public
- **ACL - access control list** -- 用户名 和 访问类型
- **FCB -- file control block**
- 目录实现：线性表，哈希表
- 分配方法(磁盘数据块分给文件)：
  - **连续分配** -- 开始块和长度in FCB; 随机访问; **外部碎片**
  - **链接分配** -- first last pointer in FCB; next pointer in Block; 无随机访问
    - **FAT MS-DOS -- 分区放file Allocation table** 含所有指针
  - **索引分配** -- 所有指针在某个**数据块**里面
    - 随机访问
    - 无外部碎片
- **空闲空间管理--位向量 链表**